CYBERSECURITY PROJECT / APPLIED CRYPTOGRAPHY

Lightweight Feistel-Based Smart Meter Encryption Suite

An 8-bit Feistel cipher for resource-constrained residential smart meter telemetry

DOMAIN	Symmetric-key cryptography, smart grid security
PRIMITIVE	Custom 8-bit Feistel network (DES-derived structure)
TARGET PLATFORM	Low-power microcontroller (8/32-bit MCU class)
DOCUMENT	Architecture, tech stack and development process

01 PROJECT OBJECTIVE

Objective and Scope

This project designs and evaluates a custom, resource-constrained 8-bit Feistel cipher for encrypting kilowatt-hour usage reports transmitted from residential smart meters to a utility provider. The cipher follows the structural principles of the Data Encryption Standard (DES), a balanced Feistel network with round-based substitution and permutation, but is scaled down from a 64-bit block to an 8-bit block so that it can run on low-power microcontrollers with limited RAM, flash, and clock speed.

The project has three measurable outcomes: a working simulation of the cipher, a measured comparison of its resource cost against a full-size reference cipher, and a statistical evaluation of its diffusion and confusion properties using the avalanche effect.

Success criteria

- Correct encryption and decryption round-trip for arbitrary 8-bit plaintext blocks.
- Measured avalanche effect close to 50% bit-flip probability on output, per NIST guidance on block cipher randomness testing.
- Execution time and estimated energy per encrypted block reported for the target microcontroller.
- A documented comparison against DES / AES on the same hardware class, explaining the security-versus-cost trade-off.

02 BACKGROUND

Cryptographic Background

Feistel structure

A Feistel cipher splits a block into two halves, L and R. In each round, the right half is passed through a round function F, keyed by a round subkey, and XORed into the left half; the halves are then swapped. This structure is reversible regardless of what F computes, because decryption simply runs the same rounds with subkeys in reverse order. DES uses a 64-bit block split into two 32-bit halves across 16 rounds; this project mirrors that logic at 8-bit scale, splitting the block into two 4-bit nibbles.

Design adaptation: 8-bit vs 64-bit DES

Property	DES (reference)	8-bit Feistel (this project)
Block size	64 bits (two 32-bit halves)	8 bits (two 4-bit halves)
Key size	56 bits effective	8 to 16 bits, configurable
Rounds	16	4 to 8, tunable for the evaluation
Round function F	Expansion, S-boxes, permutation	Small S-box (4-bit to 4-bit) plus fixed permutation
Key schedule	PC-1 / PC-2 permuted choice	Simple bit-rotation subkey derivation
Target use	General-purpose data encryption	Short, periodic telemetry values (kWh readings)

The 8-bit block size is a deliberate constraint, not an oversight: it matches the size of a single quantized usage sample and keeps every round function operating on a 4-bit S-box that fits in a handful of

microcontroller instructions. The trade-off is a much smaller key and block space, so the project treats this cipher as a case study in lightweight cryptography rather than a production-grade replacement for DES or AES.

Why a smart meter needs this trade-off

Smart meters send small, frequent, low-entropy readings (a kWh counter changes slowly and predictably) from devices with tight energy and silicon budgets. Full AES-128 is well within reach of most modern meter MCUs, but this project isolates the specific question of how far a block cipher can be shrunk before diffusion collapses, so that the resulting numbers (cycles, energy, avalanche percentage) can be compared directly against a known baseline.

03 SYSTEM ARCHITECTURE

System Architecture

The system is organized into four layers: sensing and quantization, the encryption module, the communication layer, and the utility-side decryption and validation service. The cipher itself is isolated behind a narrow interface so it can be benchmarked independently of the transport mechanism.

Layer 1: Metering and quantization

- Analog front end reads instantaneous current and voltage, computes energy over a fixed interval (for example, every 15 minutes).
- The running kWh value is quantized/truncated to an 8-bit representation suitable for the cipher's block size (with a higher-resolution value logged locally, unencrypted, for the meter's own display).
- A monotonic sequence counter and timestamp are attached alongside the encrypted block to prevent replay.

Layer 2: Encryption module (core of the project)

- Key schedule generator: derives N round subkeys from a provisioned master key.
- Feistel round engine: performs the split, S-box substitution, permutation, XOR, and swap for N rounds.
- Mode wrapper: since each block is only 8 bits, readings are chained using CBC-style XOR with the previous ciphertext block (or a nonce-derived IV per message) to avoid identical readings producing identical ciphertext.
- Self-test hook: on boot, the module encrypts a known test vector and checks it against an expected ciphertext to catch corruption or key-loading faults before any real reading is sent.

Layer 3: Communication

- Payload format: [meter ID][sequence number][timestamp][ciphertext block(s)][short integrity tag].
- Transport: simulated low-power link (for example, an emulated LoRa / Zigbee / RS-485 style serial channel appropriate to the smart-grid context) carrying the payload to a collector or gateway.
- The integrity tag is a lightweight keyed checksum over the payload, kept separate from the encryption keys, since the 8-bit cipher alone is not treated as an authentication mechanism.

Layer 4: Utility-side processing

- Gateway or head-end server holds meter keys (or per-meter derived keys from a master key plus meter ID).

- Decryption reverses the Feistel rounds using subkeys in reverse order, reconstructs the kWh value, and checks the sequence counter to reject replays or reordering.
- Decrypted readings are logged to a data store for billing and analytics; malformed or failed-integrity messages are flagged rather than silently dropped.

Data flow summary

Sensor reading → quantize to 8 bits → Feistel encrypt (N rounds, chained across messages) → attach sequence number, timestamp, integrity tag → transmit → gateway receives → verify integrity and sequence → Feistel decrypt → store reading. The evaluation harness taps this pipeline at three points: before encryption, after encryption (for diffusion analysis), and around the encryption call (for timing and power measurement).

04 TECHNOLOGY STACK

Technology Stack

Layer	Component	Tooling
Cipher reference model	Algorithm design and correctness proofing	Python 3 (bitwise operations, no external crypto libraries, so internals stay visible)
Cipher firmware implementation	Deployable round function and key schedule	C, compiled with avr-gcc or arm-none-eabi-gcc depending on target MCU
Target hardware (simulated or physical)	Execution environment for timing and power figures	ATmega328P (Arduino Uno class) or ESP32, chosen for documented current-draw datasheets
Hardware simulation	Running firmware without physical hardware on hand	Wokwi simulator or Proteus VSM, both able to trace pin activity and estimate timing
Power / energy estimation	Deriving energy-per-block from current and voltage traces	INA219 current sensor (if physical board is used) or datasheet-based current model in simulation
Timing instrumentation	Cycle-accurate or millisecond-accurate execution timing	Hardware timers (Timer0/Timer1 registers) or micros()/millis() on Arduino-class boards
Communication layer	Payload framing and transmission simulation	Serial (UART) link between two boards, or a software-simulated LoRa/Zigbee frame in Python
Statistical evaluation	Avalanche effect, bit-independence, key sensitivity tests	Python 3 with NumPy for bit-difference counting and pandas for result aggregation
Visualization	Diffusion heatmaps, power/time comparison charts	Matplotlib for static plots used in the final report
Version control and reporting	Source management and writeup	Git/GitHub for code history, LaTeX or Markdown for the final report

Why these choices

- Python is used only for the reference model and analysis, never for the timed benchmark, so measured timing reflects real embedded execution rather than an interpreter.

- C is used for the deployable cipher because it compiles to a small, predictable instruction footprint on 8-bit AVR targets, which matters when comparing cycle counts.
- Wokwi is preferred over a physical board for early development since it removes hardware-availability risk while still modeling an AVR core accurately; a physical ATmega328P or ESP32 is used for the final power measurements to validate the simulated figures.
- NumPy-based avalanche testing is chosen over a from-scratch statistical script so the diffusion analysis follows a well-understood, reproducible method rather than a bespoke one.

05 CIPHER SPECIFICATION

Cipher Design Specification

Parameters

Parameter	Value	Notes
Block size	8 bits	Split into two 4-bit halves, L0 and R0
Key size	16 bits (baseline)	Also tested at 8 and 24 bits for comparison
Rounds	4 (baseline), 6 and 8 tested	More rounds trades speed for stronger diffusion
S-box	One fixed 4-bit to 4-bit substitution table	Chosen for a balanced input/output XOR distribution
Permutation	Fixed 4-bit bit-permutation after substitution	Spreads single-bit changes across the nibble
Key schedule	Left-rotate the key by a round-dependent amount, then truncate to 4 bits per round	Deliberately simple, matching the lightweight design goal

Round function

For round i, with left half L and right half R:

```
F(R, Ki) = Permute( SBox( R XOR Ki ) )
L_new = R
R_new = L XOR F(R, Ki)
```

After the final round, the halves are not swapped, matching standard Feistel construction, so that decryption can reuse the identical round structure with the key schedule reversed.

Chaining across readings

Because an 8-bit block is small enough that repeated identical readings would otherwise produce identical ciphertext, each block is XORed with the previous ciphertext block (or an initial nonce for the first message in a session) before encryption, following the same logic as cipher block chaining. This is treated as a mitigation layered on top of the core cipher, not a substitute for a larger block size.

06 EVALUATION METHODOLOGY

Evaluation Methodology

Metric 1: Power consumption

- Measure current draw during an encryption call using an INA219 sensor (physical setup) or the simulator's current model.
- Compute energy per block as voltage x current x execution time, reported in microjoules.
- Repeat for 4, 6, and 8 rounds, and for a reference AES-128 and DES implementation on the same MCU, to produce a direct energy-per-block comparison.

Metric 2: Execution time

- Wrap the encryption call with hardware timer reads before and after, averaged over at least 1,000 calls to reduce jitter.
- Report cycles per round and total cycles per block, alongside wall-clock time at the MCU's clock speed.
- Compare against the same reference ciphers under identical timer instrumentation.

Metric 3: Ciphertext diffusion (avalanche effect)

- For a large sample of random plaintext/key pairs, flip a single input bit and count how many output bits change.
- A well-diffusing cipher should show close to 50% of output bits changing on average; report the mean and standard deviation across the sample.
- Repeat the test holding the key fixed and varying plaintext, and holding plaintext fixed and varying the key, to separate confusion from key sensitivity.
- Produce a bit-position heatmap showing which output bits are more or less sensitive to input changes, to identify weak spots in the S-box or permutation choice.

Metric 4: Security-cost trade-off summary

The final report combines all three measurements into a single comparison table across round counts and against DES/AES, so the conclusion is stated in terms of a measured trade-off (for example, an X percent reduction in energy per block against a Y percentage-point drop in average avalanche score) rather than an unqualified claim that the lightweight cipher is sufficient for production use.

07 DEVELOPMENT PROCESS

Development Process

Stage 1: Algorithm design

Define block size, key size, S-box, permutation, and key schedule on paper. Hand-verify encryption and decryption on a small worked example before writing any code.

Stage 2: Reference model in Python

Implement the cipher in Python with no external crypto libraries. Build a unit test suite covering round-trip correctness, edge cases (all-zero and all-one blocks), and known-answer test vectors generated by hand.

Stage 3: Statistical evaluation of the design

Run avalanche and key-sensitivity tests against the Python reference model first, before committing to firmware, so the S-box or round count can still be adjusted cheaply.

Stage 4: Firmware implementation

Port the finalized design to C for the target MCU. Re-run the same known-answer test vectors on the firmware output to confirm the port is bit-exact with the Python reference.

Stage 5: Communication layer integration

Wire the encryption module into a simulated meter-to-gateway message flow, including sequence numbers, timestamps, and the integrity tag.

Stage 6: Instrumentation

Add timer-based measurement around the encryption call and, where physical hardware is used, attach the current sensor. Validate instrumentation against a known workload before trusting cipher measurements.

Stage 7: Benchmarking

Run the full metric suite (power, timing, diffusion) across round counts and against the DES/AES baseline. Store raw results alongside the scripts that generated them for reproducibility.

Stage 8: Analysis and report

Aggregate results into comparison tables and plots, and write the final report stating the measured trade-offs and the conditions under which the lightweight cipher would or would not be an appropriate choice.

Testing checklist

- Round-trip test: decrypt(encrypt(x, k), k) == x for a large random sample of blocks and keys.
- Known-answer test: fixed input/key pairs with pre-computed expected ciphertext, checked on every build.
- Boundary test: all-zero block, all-one block, all-zero key, and minimum/maximum kWh quantization values.
- Cross-implementation test: Python reference and C firmware must produce identical ciphertext for the same test vectors.
- Replay test: gateway correctly rejects a retransmitted message with a stale sequence number.

08 LIMITATIONS

Known Limitations

- An 8-bit block has only 256 possible plaintexts, so without chaining, identical readings are trivially distinguishable; the CBC-style chaining mitigates but does not fully remove this limitation.
- A short key (8 to 24 bits) is exhaustively searchable on ordinary hardware in well under a second; this cipher is a lightweight-cryptography case study, not a production recommendation for protecting billing data on its own.
- The round function uses a single fixed S-box rather than the multiple distinct S-boxes DES uses per round, which is expected to reduce diffusion quality; this is measured directly rather than assumed.
- Integrity and replay protection are handled outside the cipher (sequence numbers, keyed checksum); the Feistel cipher itself provides confidentiality only.

These limitations are stated explicitly because the project's value is in measuring where a minimal Feistel design breaks down, not in proposing it as a drop-in replacement for AES in a real utility deployment.